

Developing a Reproducible, Interactive Statistics Course

Jupyter notebooks, Git, Colab, VS Code, Overleaf, HPC, and AI-supported course maintenance

Daan Meerburg

University of Groningen

Academic Staff Presentation

What I Built: Executable Course Material

- New statistics course for third-year BSc Physics students
- 12 Jupyter lecture notebooks: text, equations, code, figures, exercises
- Students run the same file they read in class
- Colab for immediate access; VS Code/Git for persistent local editing
- Board notes, quizzes, assignments, projects, exams, rubrics
- HPC development workflow synchronized with Git and Overleaf

Central choice

Jupyter notebooks

Design principle

Keep statistical theory, Python implementation, output, and interpretation together.

In class

Students can rerun and modify examples while the statistical idea is discussed.

Two options for students to run notebooks

Option 1: Colab

- click the Colab badge
- run cells immediately
- many common Python libraries already available
- use Drive/download/GitHub export for lasting copies

Option 2: VS Code + Git

- Fork or clone the repository
- Edit notebooks locally
- Commit and maintain a personal version
- More package setup, more flexibility

Trade-off

Colab is easiest for class use, but runtime state is temporary. VS Code and Git require local setup, but give students a persistent fork they can edit and maintain.

Lectures 1–4

- Introduction
- Probability
- Distributions
- Correlations

Lectures 5–7

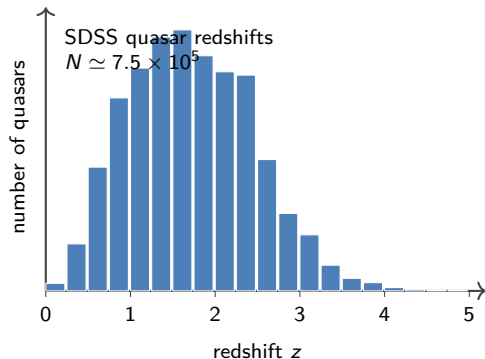
- Likelihood functions
- Maximum likelihood estimation (MLE)
- Bootstrap and jackknife
- Outliers and model comparison

Lectures 8–12

- Priors and posteriors
- Maximum a posteriori (MAP) estimation
- Marginalization
- Gibbs sampling
- MCMC and nested sampling

foundations → frequentist inference → Bayesian inference

Real Data Examples



Students load the quasar-redshift data and can change the binning, cleaning, or model.

Examples used in course/projects

- SDSS quasar redshifts
- Planck 2018 CMB bandpowers
- Pantheon+ supernova data
- BOSS/eBOSS BAO distances
- XENON1T dark-matter data
- GW170817 posterior samples
- CERN/LHC Higgs open data

Teaching use

- Plots are tied to data
- Assumptions can be changed
- Students can rerun the analysis

Student Projects

Project structure

- Timeline: 6 lecture weeks, then 2 project weeks
- Max. 5 students per project
- Student-facing project template, refined with AI support
- Notebook analysis plus final report
- Aim: a small research-style statistics analysis

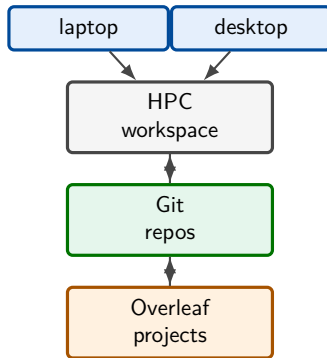
Git and assessment

- Students push notebooks and reports to Git
- Pushes and pull requests make contributions visible
- Report piloted as pass/fail
- Exam includes one project question: parts (a)–(d)
- (a,b): report literacy; (c,d): understanding

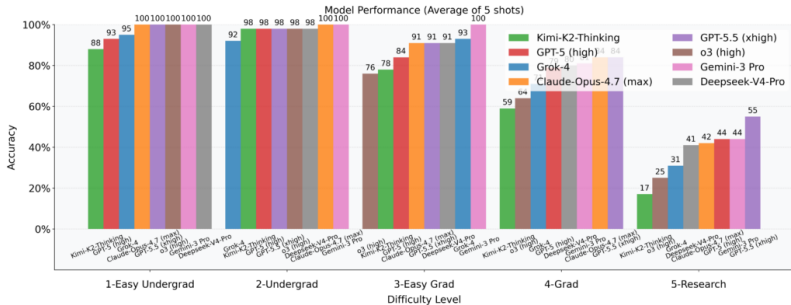
Why use HPC for Development?

My development workspace is centered on the HPC:

- Accessible from all machines
- Course repository cloned and version controlled
- Overleaf projects linked through Git
- Notebooks, notes, assignments, exams, and rubrics in one workflow
- Easy to maintain and update the course for future years



Why Try AI in This Course?



TPBench

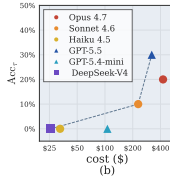
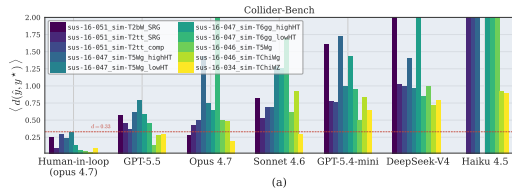
Theoretical-physics problems from undergraduate to research level.

Takeaway

GPT-5.5 is among the strongest models, with performance even on research-level problems. Undergraduate-level problems are close to saturated.

Source: TPBench, May 2026.

Scientific Coding Agents Are Being Benchmarked



Collider-Bench

Agents reproduce LHC analyses using public papers and open software.

Takeaway

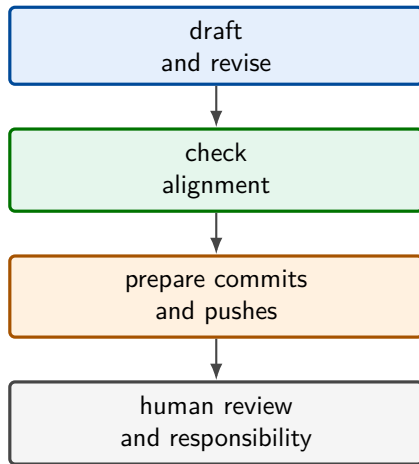
GPT-5.5 is competitive, but the paper still finds a gap to physicist-in-the-loop workflows.

Source: Faroughy et al., arXiv:2605.13950, Fig. 2.

Where AI Fit

I used AI for specific course-development tasks:

- Revise notebook explanations
- Turn notebook material into board-style notes
- Draft assignments, projects, exams, and rubrics
- Check consistency across course components
- Maintain repositories: diffs, commits, pushes, public/private separation
- List is growing as new AI tools become available and improve



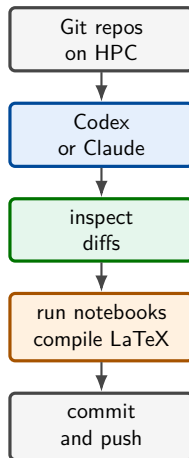
AI-Supported Course Production

From one source to many outputs

- Lecture notebooks
- Board notes in LaTeX
- Tutorial quizzes
- Assignments and projects
- Exams and rubrics

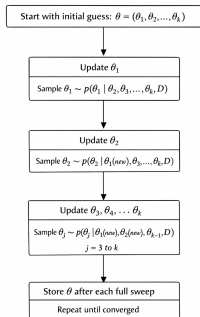
Design goal

Keep explanation, practice, project work, and assessment aligned.



AI helps draft and maintain material; release still depends on visible diffs, reruns, compilation, and instructor review.

Inference Examples in Notebooks



Gibbs sampling

Dynamic Nested Sampling with dynesty 5

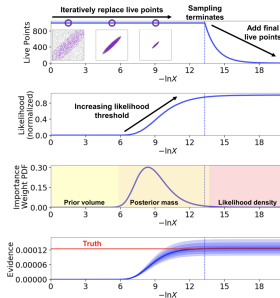


Figure 2. An example highlighting the behavior of a Static Nested Sampling run in *dynesty*. See [5] for additional details. Top: The number of live points as a function of prior volume X . Snapshots of their distribution (purple) with respect to the current bounds (grey) are [6, 1] are highlighted in several insets. The number of live points remains constant until sampling terminates, at which point we add the final live points one-by-one to the samples. Top middle: The (normalized) likelihood limit, $L_i^{(nov)}$, associated with a live point volume $\delta L_i^{(nov)}$ in the top inset. This increases monotonically as we sample increasingly smaller regions of the prior. Bottom middle: The importance weight PDF $p(X)$, roughly divided into regions dominated by the prior volume (left) in large, (left) is small; yellow), posterior mass (left and (left) are comparable, orange), and likelihood density (left is small, (left) is large, red). The posterior mass is the most important for posterior estimation, while evidence estimation also depends on the prior volume. Bottom: The estimated evidence $Z(X)$ (blue line) and its 1, 2, and 4-sigma means (blue dashed). The true value is shown in red.

not

Nested sampling

Benefits for Teaching

- Students can rerun computational examples.
- Exercises become closer to real research practice.
- Lectures, assignments, projects, and exams stay aligned.
- Updates are easier because the course is version controlled.
- Colleagues can inspect, reuse, or adapt parts of the course.

reproducibility

editable copies

maintainability

research practice

Things to keep in mind

Challenges

- Colab sessions are convenient but temporary.
- VS Code/Git gives flexibility, but needs setup.
- Notebooks can hide complexity.
- AI can produce plausible errors.

Mitigations

- Explicit persistence/forking instructions (lecture 1)
- During lecture I combine blackboard presentation with live notebook execution
- Students are encouraged to read and understand the code, not just run it
- AI is used as a drafting tool, but all changes are reviewed before release
- Private repos for exams and solutions, public repos for lectures and assignments

Take-Home Message

The course is built around version-controlled Jupyter notebooks. It uses many modern tools to make the material interactive, reproducible, and maintainable. AI integration supports this approach. The course is still in its first year, but the design principles and workflow are already showing benefits for students and for me as an instructor.

- Students can read, run, fork, and modify the lectures.
- They can choose Colab for access or VS Code/Git for persistence.
- Git, Overleaf, and the HPC make the material easier to maintain.
- AI helps with drafting and repository maintenance, but changes are reviewed before release.